



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 4: Algoritmos de exclusión mutua

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 10 de septiembre de 2026

Índice general

Índice de figuras	III
1. Solución con una variable cerrojo	1
2. Alternancia estricta entre dos procesos	3
3. Arreglo de banderas	5
4. Arreglo de banderas con orden diferente	7
5. Solución de Peterson para dos procesos	9
6. Algoritmo de la panadería de Lamport	11
7. Tabla comparativa y conclusiones	13
Referencias	15

Índice de figuras

1.1. Código fuente y captura de ejecución de la variable cerrojo. .	2
2.1. Código fuente y captura de ejecución de la alternancia estricta.	4
3.1. Código fuente y captura de ejecución de las banderas activar- comprobar.	6
4.1. Código fuente y captura de ejecución de las banderas comprobar-activar.	8
5.1. Código fuente y captura de ejecución del algoritmo de Peterson.	10
6.1. Código fuente y captura de ejecución del algoritmo de la panadería de Lamport.	12

1. Solución con una variable cerrojo

Implementa en Java una solución con una variable compartida llamada cerrojo. Utiliza al menos dos hilos, un contador compartido dentro de la sección crítica y analiza si la solución cumple exclusión mutua, progreso y espera limitada.

Entradas. Dos hilos identificados como $P0$ y $P1$, además de tres iteraciones por hilo para solicitar acceso a la sección crítica.

Procesamiento. Cada proceso comprueba si la variable cerrojo vale cero. Después de la comprobación asigna uno, ejecuta la sección crítica incrementando el contador compartido y finalmente vuelve a asignar cero para liberar el acceso.

Salidas. El orden de entrada de los hilos, el contador esperado y el contador obtenido. La prueba controlada puede producir seis incrementos esperados y solo tres obtenidos porque ambos hilos leen el mismo valor.

Estructuras de control utilizadas. Dos hilos Thread, una sección de entrada con un ciclo while, la sección crítica con un ciclo for para repetir la prueba, la sección de salida que libera el cerrojo y la sección restante. También se utilizan condicionales if y operaciones de asignación sobre variables compartidas.

Conceptos. La sección crítica modifica una variable compartida. La comprobación de cerrojo y su modificación son dos operaciones separadas; un cambio de contexto entre ambas permite que dos procesos observen cero y entren a la sección crítica. El ejemplo muestra una condición de carrera.

Prueba de escritorio.

Ejercicio 1. Solución con una variable cerrojo

Paso	Proceso	cerrojo	Acción y resultado
1	P0	0	while(cerrojo == 1) es falso; P0 puede continuar.
2	P0	0	Cambio de contexto antes de cerrojo = 1; P0 es interrumpido.
3	P1	0	while(cerrojo == 1) también es falso; P1 puede continuar.
4	P1	0	Ejecuta cerrojo = 1 y bloquea el acceso para los siguientes procesos.
5	P1	1	Entra a la sección crítica.
6	P1	1	Cambio de contexto; regresa la ejecución a P0.
7	P0	1	Continúa después del while; no vuelve a comprobar el cerrojo.
8	P0	1	Ejecuta cerrojo = 1; el valor permanece en uno.
9	P0	1	Entra también a la sección crítica; se viola la exclusión mutua.

Evaluación. No garantiza exclusión mutua porque comprobar y modificar cerrojo son dos operaciones separadas. Un cambio de contexto entre ambas permite que dos procesos entren a la sección crítica. Tampoco garantiza progreso ni espera limitada.

Explicación. La variable compartida indica si la sección crítica está libre u ocupada, pero el algoritmo no evita que un cambio de contexto ocurra entre la comprobación y la modificación. Por eso dos procesos pueden entrar a la sección crítica al mismo tiempo y no se cumple la exclusión mutua.

```

04 Algoritmo de exclusión mutua > code > java > EjercicioVariableCerrojo.java > EjercicioVariableCerrojo > proceso0()
1 // Algoritmo de exclusión mutua con una variable compartida
2 import java.util.concurrent.locks.CyclicBarrier;
3
4 public class EjercicioVariableCerrojo {
5     private static final int ITERACIONES = 5;
6     private static volatile int cerrojo = 0;
7     private static volatile int contador = 0;
8     private static final CyclicBarrier barreraCompartida = new CyclicBarrier(participantes: 2);
9
10    // Simula trabajo dentro de la sección crítica
11    private static void pasa() {
12        try {
13            Thread.sleep(1000);
14        } catch (InterruptedException e) {
15            Thread.currentThread().interrupt();
16        }
17    }
18
19    // Implementa el algoritmo de variable cerrojo
20    private static void proceso(int id) {
21        for (int iteracion = 1; iteracion == ITERACIONES; iteracion++) {
22            while (cerrojo == 1) {
23                Thread.yield();
24            }
25
26            // Fuerza el interleaving que muestra la falla del algoritmo
27            try {
28                Thread.sleep(100);
29            } catch (InterruptedException e) {
30                Thread.currentThread().interrupt();
31            }
32            return;
33        } catch (InterruptedException e) {
34            return;
35        }
36
37        cerrojo = 1;
38        System.out.println("P" + id + " entra a la sección crítica en la iteración " + iteracion);
39        int valorLeído = contador;
40        Thread.sleep(100);
41        contador = valorLeído + 1;
42        System.out.println("P" + id + " sale con contador = " + contador);
43        cerrojo = 0;
44    }
45
46    public static void main(String[] args) throws InterruptedException {
47        Thread p0 = new Thread(() -> proceso(0), "P0");
48        Thread p1 = new Thread(() -> proceso(1), "P1");
49
50        System.out.println("Prueba de variable cerrojo");
51        p0.start();
52        p1.start();
53        p0.join();
54        p1.join();
55
56        System.out.println("Contador esperado: " + (ITERACIONES * 2));
57        System.out.println("Contador obtenido: " + contador);
58        System.out.println("Resultado: comparar y modificar cerrojo son dos operaciones separadas");
59    }
60

```

```

Prueba de variable cerrojo
P0 entra a la sección crítica en la iteración 1
P0 sale con contador = 1
P1 entra a la sección crítica en la iteración 2
P1 sale con contador = 2
P0 entra a la sección crítica en la iteración 3
P0 sale con contador = 3
P1 entra a la sección crítica en la iteración 4
P1 sale con contador = 4
Contador esperado: 4
Contador obtenido: 5
Resultado: comparar y modificar cerrojo son dos operaciones separadas

```

Figura 1.1: Código fuente y captura de ejecución de la variable cerrojo.

2. Alternancia estricta entre dos procesos

Implementa la alternancia estricta entre dos procesos utilizando una variable compartida *turn*. Cada proceso debe esperar su turno, ejecutar la sección crítica y ceder el turno al otro proceso.

Entradas. Dos procesos implementados con hilos Thread, dos identificadores y tres iteraciones por proceso.

Procesamiento. El proceso P_i espera mientras *turn* sea distinto de su identificador. Cuando obtiene el turno entra a la sección crítica, incrementa el contador y, al salir, asigna el turno al proceso contrario.

Salidas. La secuencia alternada de entradas y salidas, junto con el contador esperado y el contador obtenido.

Estructuras de control utilizadas. Hilos Thread, sección de entrada con ciclo *while* para esperar, sección crítica con ciclo *for*, sección de salida que cambia *turn* y sección restante. También se utiliza una operación condicional para calcular el siguiente proceso.

Conceptos. La alternancia estricta impone un orden fijo. Si ambos procesos solicitan entrar continuamente, el orden evita que estén juntos en la sección crítica. Sin embargo, un proceso puede quedar esperando aunque el otro permanezca en su sección restante y no tenga interés en entrar; por eso falla el progreso.

Prueba de escritorio.

Ejercicio 2. Alternancia estricta entre dos procesos

Paso	Proceso	turn	Acción y resultado
1	P0	0	turn != 0 es falso; P0 puede entrar.
2	P0	0	P0 entra a la sección crítica.
3	P1	0	turn != 1 es verdadero; P1 espera.
4	P0	0	P0 termina su sección crítica.
5	P0	1	P0 ejecuta turn = 1.
6	P1	1	turn != 1 es falso; P1 puede entrar.
7	P1	1	P1 entra a la sección crítica.
8	P0	1	turn != 0 es verdadero; P0 espera.
9	P1	1	P1 termina su sección crítica.
10	P1	0	P1 ejecuta turn = 0.
11	P1	0	P1 quiere volver a entrar, pero turn != 1 es verdadero.
12	P0	0	P0 permanece en la sección restante y no quiere entrar.
13	P1	0	P1 continúa esperando aunque la sección crítica está libre.

Evaluación. Cumple exclusión mutua porque solo el proceso cuyo identificador coincide con turn puede avanzar. No cumple progreso: un proceso puede bloquear al otro aunque permanezca en su sección restante. La espera es limitada cuando los dos procesos participan de manera continua, porque el turno se alterna.

Explicación. La solución es sencilla y evita la entrada simultánea, pero su decisión depende de un proceso que puede no querer entrar. Por eso la alternancia estricta es demasiado restrictiva para una solución general.

The image shows a screenshot of an IDE with two panels. The left panel displays the source code for a Java program named 'Ejercicio2AlternanciaEstricta.java'. The code implements a strict alternance algorithm using a 'turn' variable and a 'contador' (counter) to manage two processes, P0 and P1. The right panel shows the output of the program's execution, which includes the output of the 'main' method and the 'run' method of the 'Proceso' class, demonstrating the strict alternance between the two processes.

Figura 2.1: Código fuente y captura de ejecución de la alternancia estricta.

3. Arreglo de banderas

Implementa una solución con un arreglo de banderas: primero activa la bandera y después comprueba la bandera del otro proceso antes de entrar a la sección crítica.

Entradas. Dos procesos implementados con hilos Thread y dos posiciones booleanas, una por proceso.

Procesamiento. Cada proceso, en su sección de entrada, asigna true a su bandera y luego espera mientras la bandera del otro proceso también sea verdadera. La sección crítica incrementa el contador y la sección de salida desactiva la bandera.

Salidas. La prueba muestra que las dos banderas pueden quedar activas y que ningún proceso alcanza la sección crítica. El contador permanece en cero.

Estructuras de control utilizadas. Hilos Thread, condicionales if, ciclo while para esperar, sección crítica, sección de salida y una cuenta regresiva auxiliar para reproducir la situación en la que ambos procesos esperan.

Conceptos. Las banderas expresan la intención de entrar, pero no existe un criterio para decidir quién gana cuando las dos intenciones aparecen al mismo tiempo. Ambos procesos pueden quedar en la sección de entrada y esperar indefinidamente; por eso no hay progreso.

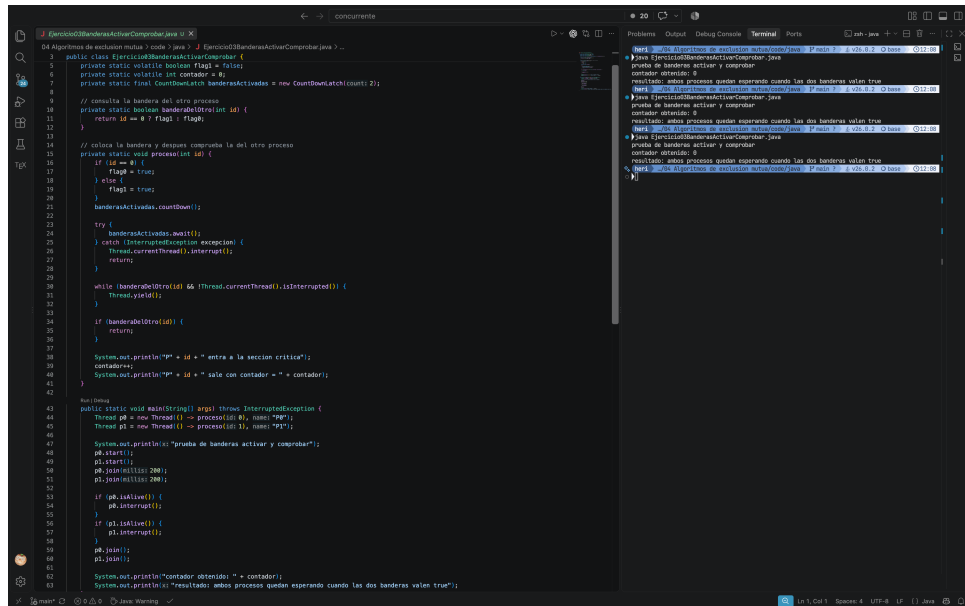
Prueba de escritorio.

Paso	Proceso	flag[0]	flag[1]	Acción y resultado
1	P0	F	F	P0 ejecuta flag[0] = true.
2	P0	T	F	Cambio de contexto antes de comprobar flag[1].
3	P1	T	F	P1 ejecuta flag[1] = true.
4	P1	T	T	while(flag[0]) es verdadero; P1 espera.
5	P0	T	T	P0 continúa después de activar su bandera.
6	P0	T	T	while(flag[1]) es verdadero; P0 espera.
7	-	T	T	Ambos esperan; ninguno entra a la sección crítica.

Evaluación. En el escenario mostrado conserva exclusión mutua porque ningún proceso entra, pero no cumple progreso ni espera limitada: ambos pueden esperar indefinidamente.

Ejercicio 3. Arreglo de banderas

Explicación. Activar la bandera antes de comprobar la bandera contraria hace que los dos procesos se observen mutuamente como interesados. Ambos quedan esperando y ninguno puede entrar a la sección crítica. Falta una variable de desempate, como turn en Peterson.



The screenshot displays an IDE with two panels. The left panel shows the source code of a Java program, and the right panel shows the output of its execution.

Source Code (Left Panel):

```
1 public class Ejercicio3BanderasActivarComprobar {
2     private static volatile boolean flag = false;
3     private static volatile int contador = 0;
4     private static final CountdownLatch banderasActivadas = new CountdownLatch(2);
5
6     // consulta la bandera del otro proceso
7     private static boolean banderaDelOtro(int id) {
8         return id == 0 ? flag : !flag;
9     }
10
11     // solicita la bandera y después comprueba la del otro proceso
12     private static void procesar(int id) {
13         if (id == 0) {
14             flag = true;
15         } else {
16             flag = false;
17         }
18         banderasActivadas.countDown();
19
20         try {
21             banderasActivadas.await();
22         } catch (InterruptedException e) {
23             Thread.currentThread().interrupt();
24             return;
25         }
26
27         while (banderaDelOtro(id) && Thread.currentThread().isInterrupted()) {
28             Thread.yield();
29         }
30
31         if (banderaDelOtro(id)) {
32             return;
33         }
34
35         System.out.println("P" + id + " entra a la sección crítica");
36         contador++;
37         System.out.println("P" + id + " sale con contador = " + contador);
38     }
39
40     public static void main(String[] args) throws InterruptedException {
41         Thread p0 = new Thread(() -> procesar(0), "P0");
42         Thread p1 = new Thread(() -> procesar(1), "P1");
43
44         System.out.println("prueba de banderas activar y comprobar");
45         p0.start();
46         p1.start();
47         p0.join(1000);
48         p1.join(1000);
49
50         if (p0.isAlive()) {
51             p0.interrupt();
52         }
53         if (p1.isAlive()) {
54             p1.interrupt();
55         }
56         p0.join();
57         p1.join();
58
59         System.out.println("contador obtenido: " + contador);
60         System.out.println("resultado: ambos procesos quedan esperando cuando las dos banderas valen true");
61     }
62 }
```

Execution Output (Right Panel):

```
Java Ejercicio3BanderasActivarComprobar.java
prueba de banderas activar y comprobar
contador obtenido: 0
resultado: ambos procesos quedan esperando cuando las dos banderas valen true
Java Ejercicio3BanderasActivarComprobar.java
prueba de banderas activar y comprobar
contador obtenido: 0
resultado: ambos procesos quedan esperando cuando las dos banderas valen true
Java Ejercicio3BanderasActivarComprobar.java
prueba de banderas activar y comprobar
contador obtenido: 0
resultado: ambos procesos quedan esperando cuando las dos banderas valen true
Java Ejercicio3BanderasActivarComprobar.java
prueba de banderas activar y comprobar
contador obtenido: 0
resultado: ambos procesos quedan esperando cuando las dos banderas valen true
```

Figura 3.1: Código fuente y captura de ejecución de las banderas activar-comprobar.

4. Arreglo de banderas con orden diferente

Implementa el arreglo de banderas con orden diferente: primero comprueba la bandera del otro proceso y después activa su propia bandera. Utiliza una variable compartida dentro de la sección crítica y analiza la entrada simultánea.

Entradas. Dos hilos y dos banderas booleanas compartidas.

Procesamiento. Cada proceso observa que la bandera contraria está en false, activa su propia bandera y entra sin volver a comprobar la bandera del otro proceso. La sección crítica incrementa el contador.

Salidas. En la prueba controlada ambos procesos entran a la sección crítica. El contador esperado es seis, pero el valor obtenido puede ser tres porque ambos pueden leer el mismo valor y escribir el mismo resultado.

Estructuras de control utilizadas. Hilos Thread, sección de entrada con ciclo while, sección crítica con ciclo for, sección de salida, condicionales if y una barrera auxiliar para alinear las comprobaciones.

Conceptos. El algoritmo tiene una condición de carrera entre la comprobación y la activación. Un cambio de contexto en ese punto permite que el otro proceso realice la misma comprobación, por lo que ambos continúan y entran a la sección crítica.

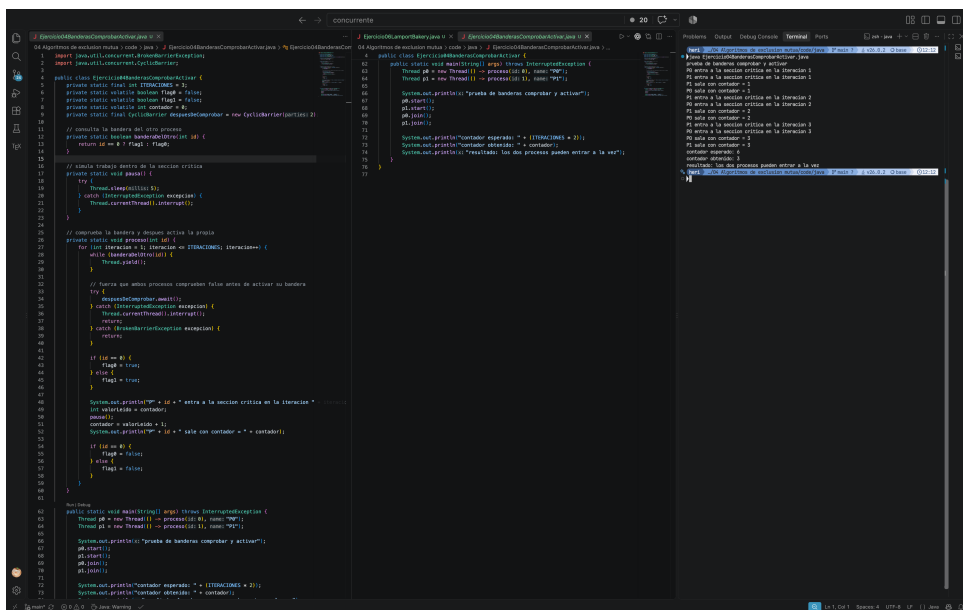
Prueba de escritorio.

Paso	Proceso	flag[0]	flag[1]	Acción y resultado
1	P0	F	F	while(flag[1]) es falso; P0 puede continuar.
2	P0	F	F	Cambio de contexto antes de activar flag[0].
3	P1	F	F	while(flag[0]) es falso; P1 puede continuar.
4	P1	F	T	P1 ejecuta flag[1] = true y entra.
5	P1	F	T	P1 se encuentra en la sección crítica.
6	P1	F	T	Cambio de contexto; regresa la ejecución a P0.
7	P0	T	T	P0 ejecuta flag[0] = true y continúa después del while.
8	P0	T	T	P0 también entra a la sección crítica; no hay exclusión mutua.

Ejercicio 4. Arreglo de banderas con orden diferente

Evaluación. No cumple exclusión mutua porque los dos procesos pueden entrar a la sección crítica al mismo tiempo. El ejemplo muestra que ambos avanzan, pero el contador compartido puede quedar inconsistente.

Explicación. El orden de las dos operaciones permite un cambio de contexto antes de que el proceso active su bandera. El otro proceso puede hacer la misma comprobación y entrar también a la sección crítica; por eso no se cumple la exclusión mutua.



```
04 Algoritmo de exclusión mutua (code > java) / Ejercicio4BanderasComprobarActivar.java % Ejercicio4BanderasComprobarActivar.java
05 import java.util.concurrent.CyclicBarrier;
06 import java.util.concurrent.locks.Lock;
07 import java.util.concurrent.locks.ReentrantLock;
08
09 public class Ejercicio4BanderasComprobarActivar {
10     private static final int ITERACIONES = 5;
11     private static volatile boolean flag = false;
12     private static volatile boolean flag2 = false;
13     private static volatile boolean flag3 = false;
14     private static CyclicBarrier barrier = new CyclicBarrier(3);
15
16     // consulta la bandera del otro proceso
17     private static boolean banderaOtroProceso() {
18         return flag & flag2 & flag3;
19     }
20
21     // simula trabajo dentro de la sección crítica
22     private static void paso1() {
23         try {
24             Thread.sleep(1000);
25         } catch (InterruptedException e) {
26             Thread.currentThread().interrupt();
27         }
28     }
29
30     // comprueba la bandera y después activa la propia
31     private static void proceso1() {
32         for (int iteracion = 1; iteracion <= ITERACIONES; iteracion++) {
33             while (banderaOtroProceso()) {
34                 Thread.sleep(100);
35             }
36
37             // entra que otros procesos comprueban false antes de activar su bandera
38             try {
39                 Thread.sleep(1000);
40             } catch (InterruptedException e) {
41                 Thread.currentThread().interrupt();
42             }
43             return;
44         }
45         if (flag == false) {
46             flag = true;
47             flag2 = true;
48             flag3 = true;
49
50             System.out.println("id = " + iteracion + " entra a la sección crítica en la iteración " + iteracion);
51             try {
52                 Thread.sleep(1000);
53             } catch (InterruptedException e) {
54                 Thread.currentThread().interrupt();
55             }
56             System.out.println("id = " + iteracion + " sale de la sección crítica en la iteración " + iteracion);
57             flag = false;
58             flag2 = false;
59             flag3 = false;
60         }
61     }
62
63     public static void main(String[] args) throws InterruptedException {
64         Thread p1 = new Thread(() -> proceso1(), "p1");
65         Thread p2 = new Thread(() -> proceso2(), "p2");
66         Thread p3 = new Thread(() -> proceso3(), "p3");
67
68         p1.start();
69         p2.start();
70         p3.start();
71
72         System.out.println("contador esperado: " + (ITERACIONES * 3));
73         System.out.println("contador obtenido: " + contador);
74         System.out.println("resultado: los dos procesos pueden entrar a la vez");
75     }
76 }
```

Output:

```
id = 1 entra a la sección crítica en la iteración 1
id = 2 entra a la sección crítica en la iteración 2
id = 3 entra a la sección crítica en la iteración 3
id = 1 sale de la sección crítica en la iteración 1
id = 2 sale de la sección crítica en la iteración 2
id = 3 sale de la sección crítica en la iteración 3
id = 1 entra a la sección crítica en la iteración 4
id = 2 entra a la sección crítica en la iteración 5
id = 3 entra a la sección crítica en la iteración 6
id = 1 sale de la sección crítica en la iteración 4
id = 2 sale de la sección crítica en la iteración 5
id = 3 sale de la sección crítica en la iteración 6
resultado: los dos procesos pueden entrar a la vez
```

Figura 4.1: Código fuente y captura de ejecución de las banderas comprobar-activar.

5. Solución de Peterson para dos procesos

Implementa el algoritmo de Peterson para dos procesos. Cada proceso debe utilizar una bandera y una variable turn, ejecutar la sección crítica y demostrar el comportamiento del contador compartido.

Entradas. Dos hilos, dos banderas, una variable turn y tres iteraciones por hilo.

Procesamiento. El proceso activa su bandera, cede la prioridad al otro con turn y espera únicamente si el otro también quiere entrar y conserva la prioridad. Al salir desactiva su bandera.

Salidas. El orden de acceso y el contador final. Para seis entradas el contador esperado y el contador obtenido coinciden.

Estructuras de control utilizadas. Hilos Thread, sección de entrada con ciclo while para esperar, sección crítica con ciclo for, sección de salida que desactiva la bandera, sección restante y condicionales para representar las decisiones entre procesos.

Conceptos. Peterson combina la intención de entrar, expresada por flag, con un desempate mediante turn. Si ambos procesos quieren entrar, solo uno puede observar una condición de espera falsa. Cuando uno sale, baja su bandera y permite que el otro continúe. Para dos procesos garantiza exclusión mutua, progreso y espera limitada.

Prueba de escritorio.

Ejercicio 5. Solución de Peterson para dos procesos

Paso	Proceso	flag[0]	flag[1]	turn	Acción y resultado
1	P0	T	F	–	P0 activa flag[0].
2	P0	T	F	1	P0 ejecuta turn = 1.
3	–	T	F	1	Cambio de contexto antes de que P0 compruebe la condición.
4	P1	T	T	1	P1 activa flag[1].
5	P1	T	T	0	P1 ejecuta turn = 0.
6	P1	T	T	0	P1 espera porque flag[0] es verdadera y turn == 0.
7	P0	T	T	0	P0 entra: turn == 1 es falso.
8	P0	F	T	0	P0 sale y ejecuta flag[0] = false.
9	P1	F	T	0	P1 entra porque flag[0] ya es falsa.
10	P1	F	F	0	P1 sale y ejecuta flag[1] = false.

Evaluación. Cumple exclusión mutua, progreso y espera limitada para dos procesos, suponiendo que las variables compartidas sean visibles y que los procesos continúen ejecutándose.

Explicación. La bandera expresa la intención de entrar y turn resuelve el empate cuando ambos procesos quieren entrar. Así, uno espera y el otro entra; al salir, desactiva su bandera y permite que el otro continúe.

```

01 Algoritmo de exclusión mutua - code - java - J. DemocastPeterson.java - % DemocastPeterson
02
03 public class EjercicioPeterson {
04     private static boolean flag0 = false;
05     private static boolean flag1 = false;
06     private static boolean turn = 0;
07     private static boolean wait0 = false;
08     private static boolean wait1 = false;
09
10     // consulta la bandera de un proceso
11     private static boolean bandera0() {
12         return flag0 & !wait0;
13     }
14
15     // consulta la bandera de un proceso
16     private static boolean bandera1() {
17         return flag1 & !wait1;
18     }
19
20     // cambia trabajo dentro de la sección crítica
21     private static void paso0() {
22         try {
23             System.out.println("P0");
24             // crítica (interacción con el mundo exterior)
25             Thread.currentThread().sleep(100);
26         } catch (InterruptedException e) {
27             e.printStackTrace();
28         }
29     }
30
31     // aplica el algoritmo de Peterson para dos procesos
32     private static void proceso0() {
33         int otro = 1 - 0;
34
35         for (int iteracion = 0; iteracion < 10; iteracion++) {
36             System.out.println("P0: iteración " + iteracion);
37             System.out.println("P0: turn == otro? " + (turn == otro));
38             if (bandera0() & !bandera1()) {
39                 System.out.println("P0: entra a la sección crítica en la iteración " + iteracion);
40                 paso0();
41                 System.out.println("P0: contador = " + contador);
42                 contador++;
43                 System.out.println("P0: sale de la sección crítica en la iteración " + iteracion);
44                 wait0 = false;
45                 System.out.println("P0: bandera desactivada");
46             }
47         }
48     }
49
50     // aplica el algoritmo de Peterson para dos procesos
51     private static void proceso1() {
52         int otro = 1 - 1;
53
54         for (int iteracion = 0; iteracion < 10; iteracion++) {
55             System.out.println("P1: iteración " + iteracion);
56             System.out.println("P1: turn == otro? " + (turn == otro));
57             if (bandera1() & !bandera0()) {
58                 System.out.println("P1: entra a la sección crítica en la iteración " + iteracion);
59                 paso1();
60                 System.out.println("P1: contador = " + contador);
61                 contador++;
62                 System.out.println("P1: sale de la sección crítica en la iteración " + iteracion);
63                 wait1 = false;
64                 System.out.println("P1: bandera desactivada");
65             }
66         }
67     }
68
69     public static void main(String[] args) throws InterruptedException {
70         Thread p0 = new Thread() {
71             @Override public void run() {
72                 proceso0();
73             }
74         };
75         Thread p1 = new Thread() {
76             @Override public void run() {
77                 proceso1();
78             }
79         };
80         p0.start();
81         p1.start();
82         p0.join();
83         p1.join();
84
85         System.out.println("contador esperado: " + (10 * 2));
86         System.out.println("contador obtenido: " + contador);
87         System.out.println("resultado: cumple exclusión mutua, progreso y espera limitada para dos procesos");
88     }
89 }
  
```

Figura 5.1: Código fuente y captura de ejecución del algoritmo de Peterson.

6. Algoritmo de la panadería de Lamport

Implementa el algoritmo de Lamport o de la panadería para N procesos. Utiliza al menos tres hilos, números de turno y una variable compartida dentro de la sección crítica.

Entradas. Tres hilos identificados como $P0$, $P1$ y $P2$, tres iteraciones por hilo y los arreglos compartidos de selección y números.

Procesamiento. Cada proceso anuncia que está eligiendo turno, toma un número mayor que los existentes y espera a los procesos con un turno menor. En caso de empate se utiliza el identificador menor. Al salir de la sección crítica, coloca su número en cero.

Salidas. Los turnos asignados, el orden de entrada y el contador final. Para nueve entradas el contador esperado y el contador obtenido coinciden.

Estructuras de control utilizadas. Un arreglo de hilos, una sección de entrada con ciclos for para buscar el número mayor y comparar procesos, ciclos while para esperar, la sección crítica, la sección de salida que coloca el número en cero y condicionales para resolver empates.

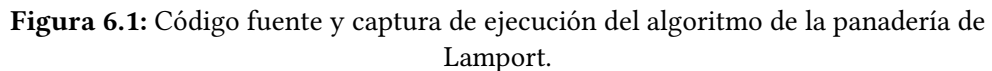
Conceptos. La panadería de Lamport simula una fila de panadería. El par formado por $\text{numero}[i]$ e i establece un orden total, equivalente a comparar primero el número y después el identificador del proceso. Los arreglos compartidos del programa representan los procesos que están eligiendo turno y los números asignados.

Prueba de escritorio.

Evaluación. Cumple exclusión mutua, progreso y espera limitada para N procesos, siempre que las variables compartidas sean visibles y cada proceso libere su número al salir.

Explicación. El número de turno ordena las solicitudes sin depender de un único proceso. Cada proceso consulta los números de los demás antes de entrar a la sección crítica y coloca su número en cero al salir.

Explicación. El número de turno ordena las solicitudes sin depender de un único proceso. Cada proceso consulta los números de los demás antes de entrar a la sección crítica y coloca su número en cero al salir.



7. Tabla comparativa y conclusiones

La tabla resume el comportamiento observado y las propiedades teóricas de las seis estrategias. La columna de procesos muestra la configuración utilizada en los programas entregados. En cada caso se distinguen la sección de entrada, la sección crítica, la sección de salida y la sección restante.

Cuadro 7.1: Comparación de algoritmos de exclusión mutua

Algoritmo	Proc.	Excl. mu- tua	Prog.	Espera limitada	Problema principal
Variable cerrojo	2	No	No ¹	No	Condición de carrera entre comprobar y asignar
Alternancia estricta	2	Sí	No	Sí ²	Depende de un proceso que puede no querer entrar
Arreglo de banderas	2	Sí ³	No	No	Ambas banderas pueden quedar activas
Banderas con orden diferente	2	No	Sí ⁴	No	Condición de carrera antes de activar
Peterson	2	Sí	Sí	Sí	Está diseñado para dos procesos
Lamport/panadería	N (3)	Sí	Sí	Sí	Más procesos y números de turno

1. La variable no es suficiente para garantizar progreso; además, el proceso puede ser adelantado repetidamente.
2. Cuando los dos procesos solicitan acceso de forma continua; si uno no participa, puede bloquearse el otro.
3. En la ejecución simultánea ninguno entra, por lo que no hay violación observada, pero no existe progreso.
4. La variante permite que ambos avancen, pero no garantiza exclusión mutua.

¿Cuál es el algoritmo más adecuado? Para exactamente dos procesos, Peterson es la opción más clara porque combina banderas y turno con una prueba sencilla de exclusión mutua, progreso y espera limitada. Para un número arbitrario de procesos, la panadería de Lamport es más adecuada porque generaliza el orden de entrada mediante números de turno.

Ejercicio 7. Tabla comparativa y conclusiones

¿Qué limitaciones se observan? En los ejemplos, los procesos pueden permanecer en un ciclo `while` mientras esperan. El cerrojo simple y las dos variantes de banderas muestran que cambiar el orden de lectura y escritura puede producir una condición de carrera o pérdida de actualizaciones. Las banderas pueden dejar a ambos procesos esperando. Peterson está limitado a dos procesos y la panadería de Lamport necesita más memoria y comparaciones a medida que aumenta el número de procesos.

Bibliografia

- [1] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed. Reading, Massachusetts: Addison-Wesley, 1994.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 7th ed. Boston, MA: Prentice Hall, 2011.